

# 共识算法

陈润宇

对外经济贸易大学信息学院



# 内容提纲

- 分布式共识概述
- 经典共识算法
- 共识算法的发展

# 共识算法简史

- 1975 年阿克云卢、埃卡纳德汉姆和胡贝尔在论文《网络通信设计中的一些制约因素和权衡问题》中首次提到了“两军问题”(Two Generals' Problem): 通信双方通过派遣信使达成一致的行动一并证明该问题无解。1978年, 吉姆·格雷在《关于数据库操作系统的说明》一文中将该问题命名为“两军悖论”。该文献被广泛用作两军问题及其不可能性证明的参考文献。
- 罗伯特·肖斯塔克、马歇尔·皮斯、莱斯利·兰伯特三人在1980年发表论文《在有缺陷的情况下达成协议》。为了使“交互式一致性”问题更容易理解, 莱斯利·兰伯特编写了一个故事, 描写一群将军制订了进攻敌对城市的计划。最早的故事使用的是阿尔巴尼亚军队, 之后杰克·戈德堡建议改名为拜占庭。1982年上面提到的三位作者发表了包含这个故事的论文《拜占庭将军问题》, 文中给出了基于口头消息和签名消息的两种解决方法。
- 1985 年迈克尔·J.费舍尔、南希·林奇、迈克·帕特森共同发表了论文《一个错误的过程不可能达成分布式共识》, 提出了FLP不可能原理, 即在完全异步的系统中, 只要有一个进程可能发生故障, 那么就无法保证在有限时间内可以使所有进程达成一致。
- 1988年布瑞恩·奥基和芭芭拉·利斯科夫共同发表论文《视图标记复制: 支持高可用分布式系统的新主复制方法》, 提出了视图标记复制(Viewstamped Replication VR)一致性算法, 采用主机-备份(Primary-Backups)模式, 所有数据操作都必须通过主机进行, 再复制到各备份机器以保证一致性。
- 1990年莱斯利·兰伯特提出了一种基于消息传递的一致性算法Paxos, 这是一种基于消息传递且具有高度容错特性的共识(Consensus)算法。Paxos算法获得了广泛的认可, 现已衍生出Paxos算法家族。因为VR和Paxos算法都不考虑可能出现消息篡改即拜占庭错误的情况, 因此这两种是非拜占庭容错算法, 也叫崩溃容错(Crash Fault Tolerant, CFT)算法。

# 共识算法简史

- 1993 年辛西娅·德沃克和莫尼·纳尔在学术论文《通过处理或打击垃圾邮件的方式定价》中提出了工作量证明的想法，可以用来解决垃圾邮件问题。一般要求用户进行一些耗时的复杂运算，并且运算结果容易验证，以此消耗的时间、设备与能源作为担保成本，以确保服务与资源是被真正的需求方所使用。PoW 为后面比特币的出现奠定了一个方面的基础。
- 实用拜占庭容错算法提供了高性能的拜占庭状态机复制，以仅仅增加毫秒级延迟的极小代价每秒处理数千个请求。实用拜占庭容错算法主要解决早期的拜占庭容错算法效率低下的问题，使得拜占庭容错算法在实际应用中变得可行。
- 在2000年的分布式计算原理研讨会(PODC)上，加州大学柏克莱分校的计算机科学家埃里克·布鲁尔提出了一个猜想，即对于一个分布式计算系统来说，不可能同时满足一致性、可用性和分区容错性，这就是后来有名的 CAP定理。2002年麻省理工学院的南希·林奇和赛斯·吉尔伯特发表了埃里克·布鲁尔(Eric Brewer)猜想的证明，使之成为一个定理。
- 2008年中本聪发表了作为比特币奠基的论文《比特币：一种点对点的电子现金系统》。之后使用 PoW 共识算法的比特币主网开始运行，从此开启了区块链共识算法研究的序幕。

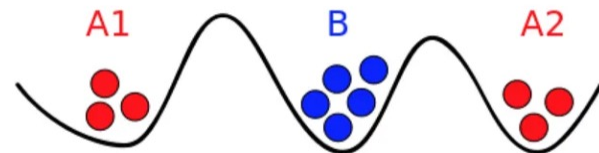
# 分布式共识问题

## ■ 共识问题

- 在分布式系统中，共识指在部分节点故障、网络延时、网络分区甚至有恶意节点故意破坏的情况下，所有正常节点对提案中的请求在系统中达成一致的过程。
- 现今共识机制常见于区块链领域。区块链共识算法建立在一系列协议或规则基础上，在对网络中的交易共识的过程中，完成记账节点选择、区块生成、验证、上链等操作，最终保持分布式账本数据的一致性。共识算法是区块链系统的核心技术，决定着区块链系统最终所能达到的性能以及可以应用的领域。

## ■ 问题假设

- 分布式系统理论研究从基本的问题假设出发，问题假设是对现实问题的简化。
- 1.两军问题
- 两军问题是计算机领域中的一个思想实验。在该问题中，假设两支军队（A1A2）的将军只能通过派遣信使穿越敌方领土（B）来互相通信，所派出的任何信使都可能被俘虏。在此条件之下，该问题希望求解如何就发动攻击的时间点达成共识。



# 分布式共识问题

- 1.两军问题
- 两军问题理论上无解，通过不可靠的信道交换信息并达成共识是不可能的，第n条消息的发送者无法确认其消息会被对方收到。两军问题是第一个被证明无解的计算机通信问题，对计算机通信领域产生了重大影响，解释了TCP协议无法保证通信双方之间的状态一致性的原因。
- TCP协议中的“三次握手”过程仅仅是对问题理论上的解，如图3-1所示。两军问题的重要意义在于，对所有存在通信错误的问题场景，包括拜占庭将军问题，都是无解的。两军问题可以看作拜占庭将军问题的一个特例。

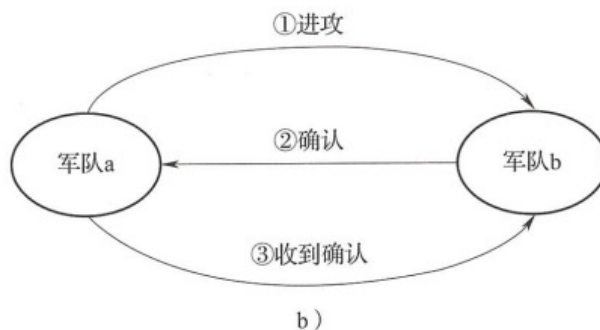


图 3-1 两军问题的消息通信

# 分布式共识问题

## ■ 2.Paxos岛议会问题

- 莱斯利·兰伯特在描述Paxos算法时，虚拟了一个叫作Paxos的希腊城邦，这个岛按照议会民主制的政治模式制定法律，但是没有人愿意将自己的全部时间和精力放在这种事情上。无论是议员、议长还是传递纸条的服务员都不能承诺别人需要时一定会出现，也无法承诺批准决议或者传递消息的时间。议员之间的消息要靠服务员传递，服务员有可能忘记传递，也有可能重复传递。一般来说只要等待足够的时间，消息就会被传到。
- Paxos算法不可避免地会发生以下错误：进程可能处理得很慢、被杀死或者重启，消息可能会延迟、丢失、重复。假设没有消息篡改的情况，即虽然有可能一个消息被传递了两次，但是绝对不会出现错误的消息。Paxos岛议会是一个非拜占庭容错的场景，如图3-2所示。

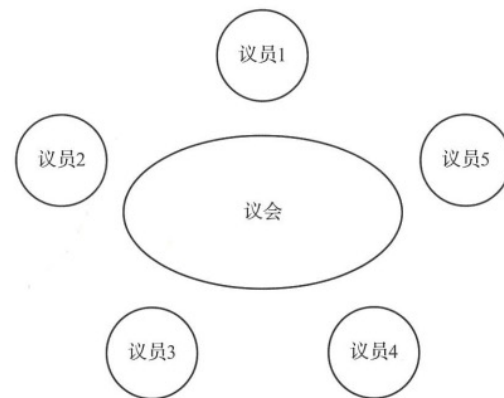


图 3-2 Paxos 岛议会

# 分布式共识问题

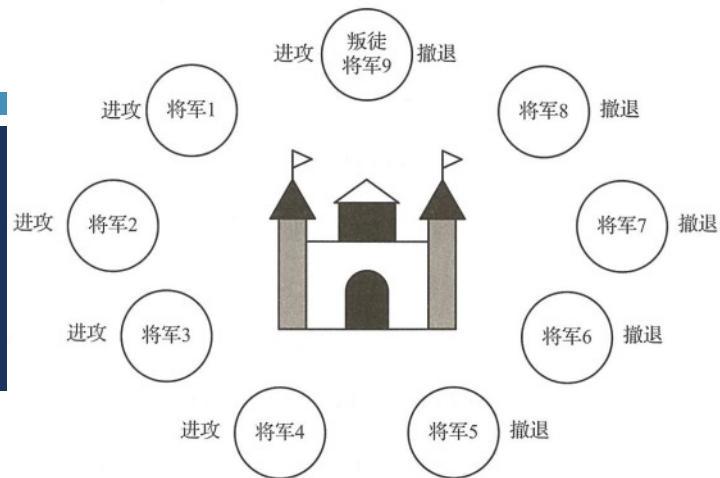


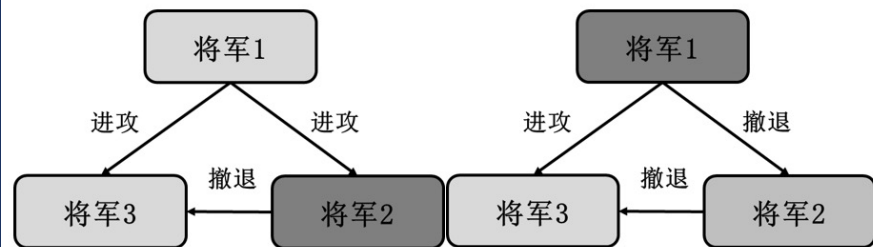
图 3-3 拜占庭将军问题

## ■ 3.拜占庭将军问题

- 一组拜占庭将军分别率领一支军队围困一座城市。为了简化问题，将各支军队的行动策略限定为进攻或撤离两种。部分军队进攻部分军队撤离可能会造成灾难性后果，因此各位将军必须通过投票来达成一致策略。
- 各位将军分处城市的不同方向，他们只能通过信使互相联系。在投票过程中每位将军都将自己投票进攻或者撤退的信息交给信使，由信使分别通知其他将军，这样一来每位将军根据自己的投票和其他将军送来的信息就可以知道共同的投票结果而决定行动策略。
- 系统的问题在于，将军中可能出现叛徒，他们不仅可能向较为糟糕的策略投票，还可能选择性地发送投票信息。由于将军之间需要通过信使通讯，叛变将军可能通过伪造信件来以其他将军的身份发送假投票。
- 而即使在保证所有将军忠诚的情况下，也不能排除信使被敌人截杀，甚至被敌人间谍替换等情况。因此很难通过保证人员可靠性及通讯可靠性来解决问题。这时候，在已知有成员谋反的情况下，其余忠诚的将军在不受叛徒的影响下如何达成一致的协议，就由此形成了历史中鼎鼎有名的拜占庭将军问题。



# 分布式共识问题



## ■ 3.拜占庭将军问题

- 如图可以快速说明在有三位将军的情况下，只要有一个叛徒存在，就不能达成一致的作战计划。深色代表叛徒，浅色代表忠诚的将军。如左图，当接收作战计划的一位将军是叛徒时，忠诚的将军将收到相悖的计划而选择不执行；如右图，当传递作战计划的将军是叛徒时，会向两位将军发送不同的作战计划，导致一位将军接收到相悖的计划而选择不执行。
- 针对拜占庭将军问题，一般有通过口头协议和通过消息签名两种解决方法。
  - “口头协议”的OM(m)协议默认了每个被发送的消息都能够被正确传递，并且消息的接收者能够知道消息的发送者和缺少的信息。在传播链路上，除加密安全保障外，不允许使用任何的加密算法。采用这种方法时，问题有解的充分必要条件是叛徒数目少于将军总数的 $1/3$ 。但是，这种方式无法实现对消息的溯源。
  - “消息签名”的SM(m)协议则有效解决了这一问题。该算法与第一种不同之处在于，使用签名算法，即每个节点都能产生一个不可伪造的签名，其他任意节点都能够对该签名进行验证。采用这种方法时，无论叛徒数目有多少，问题均是有解的。同时，该算法带来的信息传递延迟问题目前没有得到有效的解决。
- 综上所述，拜占庭将军问题的实质就是要寻找一个方法，使得在有故障的情况下能建立系统的共识。在分布式系统中，存在正常运行的节点，存在故障节点，当然也可能存在具有破坏性的节点。而共识算法的核心，就是解决拜占庭将军问题，在故障和破坏性节点存在的条件下，在正常节点之间形成统一意见与认知。

# 分布式共识问题总结

## ■什么是一致性问题？

- 一致性是指分布式系统中的多个节点经过相同的一系列操作达成相同的结果，这是分布式领域最基本的问题，也是分布式系统所要实现的目标。
- “一致性”和“共识”并不是同一个概念。文献中对共识问题与一致性问题的描述非常接近，经常用来相互解释，此二者在侧重点上有所不同，前者侧重于分布式系统达成一致性结果的过程和算法实现，后者侧重于经过共识过程所达成的稳定状态。
- 例如，在一个分布式数据库系统中，如果各节点的初始状态一致，而且每个节点所执行的指令序列相同，那么它们最终会达成一致的状态。为保证每个节点执行相同的指令序列，在执行每一条指令时都要经过“共识”。“共识”的目的就是为了确保分布式系统状态的“一致性”。

## ■容错算法

- 在分布式系统中，根据系统对故障组件的容错能力分为崩溃容错算法(Crash Fault Tolerant, CFT)和拜占庭容错算法(Byzantine Fault Tolerant, BFT)，其中关键的区别在于是否假设系统中存在拜占庭节点，即恶意节点。
- 分布式系统要达成一致性状态，是否容错拜占庭节点，其共识算法的设计大不相同。区块链系统处在一个更加开放的网络环境下，必须考虑拜占庭节点作恶的情况。“共识算法简史”部分提到的视图标记复制算法VR属于CFT算法，而实用拜占庭容错算法，工作量证明属于BFT算法。

# 分布式共识问题总结

■现实中的分布式系统都是异步系统，处于比较复杂的环境条件下，为了达成“一致性”的状态，往往要面对下面的问题挑战：

- (1)网络存在时延、吞吐率低。
- (2)信道不可靠、信息丢失。
- (3)所发出的消息不能在预期的时间内到达。
- (4)机器上的时钟、时钟漂移率精度不够。
- (5)机器宕机，进程崩溃。
- (6)节点恶意篡改数据。

■ 分布式系统所要面对的这些挑战，更深层次的问题来源有以下几个方面：

- (1)并发性(Concurrency)：多个程序(进程、线程)并发执行，共享资源。
- (2)没有全局时钟(GlobalClock)：每个机器有各自的时间没有办法做到统一，程序间的协调依靠交换消息。
- (3)故障独立性(Independent Failure)：一些进程出现故障，并不能保证其他进程都能知道。

# 区块链系统共识过程模拟

- 1.选择出块节点：所有节点通过执行共识算法实现决定哪个节点获得出块权的工作，因算法实现的不同，系统中各个节点可能扮演的角色会差别较大。
- 2.构造新区块：获得出块权的节点，按照共识算法中的打包策略，将最近一段时间全系统中新发生的交易数据按照一定的顺序打包到一个新区块中。
- 3.广播并验证新区块：出块节点按照共识算法中既定策略，将新区块广播给承担验证任务的节点，验证节点收到广播来的新区块后，验证新区块中封装的交易数据的合法性。如果新区块通过大多数验证节点的验证，则等待被加入区块链中成为最新的一个区块。
- 4.新区块上链：出块节点将通过验证的新区块添加到系统主链上，构成一条从创世区块开始的最长的链。
- 以上可以代表多数区块链的共识过程，但无法概括所有类型。区块链系统中的节点分为发起交易的普通节点和打包交易并进行验证的出块节点。区块链共识算法的核心是“选主”和“记账”两部分，前者有时候也称为选主策略。有的共识算法需要节点参加共识过程并竞争记账权，有的算法中则会选举特定的节点作为代表参与共识，为了公平有时候会按照轮次重新选举。

# 共识算法分类

- 根据不同的维度，通常有以下几种分类方法：
- 是否拜占庭容错
  - 拜占庭容错共识，如 PBFT，PoW，PoS；
  - 非拜占庭容错共识，如 VR，Paxos，Raft。
- 区块链系统部署类型
  - 公有链共识，如PoW，PoS，DPoS。
  - 联盟链共识，如PBFT，PoA，rPBFT。
- 选主策略
  - 区块链的每一轮共识中，都需要记账节点构造并发布新区块，选主策略的目的就是找出记账节点。根据这个特性，共识算法分为选举类、证明类、随机类共识。
  - 选举类共识，通过投票的方式选出下一轮的记账节点，如Paxos，Raft，dBFT。
  - 证明类共识，通过付出算力或者持有权益竞争记账权，获得收益，如PoW，PoS。
  - 随机类共识，通过随机方式确定记账节点，如阿格兰德、时间消逝证明(Proof of Elapsed Time, PoET)。

# 共识算法评估

## ■安全性和容错能力

- 这是指是否可以防止二次支付、私自挖矿等攻击，是否有良好的容错能力。

## ■可扩展性

- 这是指是否支持网络节点扩展。扩展性是区块链设计要考虑的关键因素之一。

## ■性能

- 延迟是性能指标，是指交易达成共识被记录在区块链中至被最终确认的时间，关系到系统每秒可处理确认的交易数量(Transactions Per Second, TPS)。TPS 是区块链被考虑最多的性能指标。

## ■资源消耗

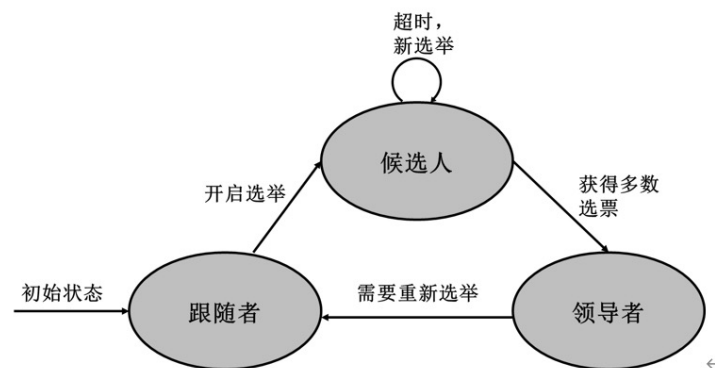
- 即在达成共识的过程中，系统所要耗费的计算资源大小，包括中央处理器(CPU)、内存等。区块链需要借助计算资源或者网络通信资源达成共识。

## ■去中心化程度

- 区块链具有分布式自治的特性，去中心化程度越高，治理过程越公正，系统越稳定，但效率相对越低;相反，去中心化程度越低，共识过程越集中，效率越高，相对容易出现造假现象。

# 经典共识算法：RAFT

- RAFT算法是2013年由迭戈·翁伽罗（Diego Ongaro）和约翰·奥斯特豪（John Ousterhout）提出的一种适用于非拜占庭容错环境下的分布式一致性算法。
- RAFT共识算法的基本流程为：首先在集群中通过选举确定领导者（Leader），全权负责管理信息的复制。领导者从客户端接收交易信息，在添加到自身的日志后，向其他节点发送该信息，并通知其他节点将信息添加到日志中。当领导者宕机或与其他节点的连接断开后，其他节点会重新启动选举选出新的领导者。
- 在RAFT共识中，共有以下三种节点状态，分别为领导者（Leader）、候选人（Candidate）和追随者（Follower）。集群中的所有节点都处在这三种状态之一。一般情况下，集群中只有一个leader，剩余节点均为follower。跟随者节点只需要被动接收来自leader的信息并给出响应，自身不发送任何请求。候选人状态出现在选举过程中。下图给出了三种状态的转换关系。

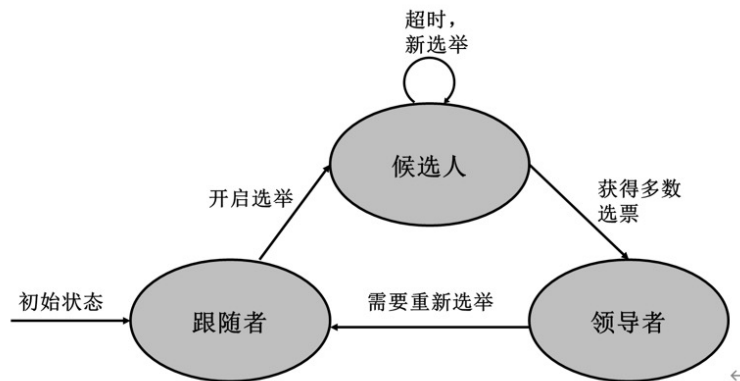


RAFT 节点状态及转换关系



# 经典共识算法：RAFT

- 在选举开始时，所有节点都维持follower状态，并获得相应时长的计时器。若在计时器超时前仍未收到其他候选人节点的投票请求，则自身变为候选人节点并邀请其他follower为自己投票；若在计时器超时前收到投票请求，则为其投票并置计时器为零。Candidate的票数超过节点半数即成为leader，其他candidate自动降为follower。一旦现有leader宕机，则会在follower中重新选举。
- RAFT算法通过设置不同的节点状态，简化了分布式系统记账的操作，但该算法只支持容错故障节点，对于拜占庭节点不具有支持性。因此，RAFT共识算法可以应用于非拜占庭容错环境，一般适用于联盟链和私有链场景。



RAFT 节点状态及转换关系



# 经典共识算法：POW

■工作量证明（Proof-of-Work, PoW）是区块链上使用最早也是最广泛的共识算法之一，其思想由来已久。关于PoW的学术研究始于1993年，辛西娅·德沃克（Cynthia Dwork）和莫尼·瑙尔（Moni Naor）在学术论文中提及这一概念。1997年，亚当·贝克（Adam Back）独立发明了基于PoW的哈希现金，用于抵抗邮件的拒绝服务攻击和垃圾邮件网关滥用。1999年，马库斯·雅各布松（Markus Jakobsson）和阿里·朱尔斯（Ari Juels）正式提出了“工作量证明”一词。

■PoW共识算法的本质是根据节点的计算能力来决定记账权。简单来说，PoW是一份证明，证明方在使用服务或资源前，需要完成具有一定难度或适当工作量的复杂运算，并且这种工作量可以很容易地被验证方核实。其核心思想是通过引入分布式节点的算力竞争来保证数据的一致性和安全性。在区块链领域，PoW共识算法最成功的应用就是比特币系统。

■2008年，中本聪的比特币创世论文的重要贡献之一就是将PoW思想应用于区块链共识过程，实现了区块链的PoW共识算法。根据比特币的设计，系统大约每10分钟生成一个区块，所有比特币矿工均参与竞争这种极度稀缺的“记账权”与比特币奖励

■PoW共识算法利用分布式节点的算力来完成大量的哈希函数计算工作，选出每10分钟的唯一“记账人”，从而保证账本数据的一致性。

# 经典共识算法：POW

- 为了更好地理解工作量证明的过程，使用一个简单的例子来进行说明。给定一个字符串“Hello, blockchain! ”，PoW过程可以理解为寻找一个合适的随机数nonce，作为哈希函数SHA256的输入进行哈希运算。如果得到的十六进制的哈希结果是以“0000”（n个0）开头的，则通过验证。
- 比特币的PoW过程与上面的例子基本一致，而每一次共识过程的困难程度则会根据难度值的输入来进行调整，以控制PoW的出块速度稳定在10分钟左右。在哈希过程中，区块头哈希值的前导零就越多，成功找到合适的随机数的难度越大。如2022年11月，符合要求的区块头哈希值需要有19个前导零，每16次随机数搜索将会找到一个含有前导零的哈希值，则19位的前导零要求16的19次随机数搜索才能找到一个合适的随机数并生成新的区块。由此可知，比特币区块链系统的安全性正是由PoW的强大算力保证的，所有针对比特币的攻击都必须对SHA256难题进行计算，这一计算过程的难度和投入是非常大的。
- PoW共识算法的优势在于其执行简单，能够通过算力竞争来有效实现保障系统安全的效果。PoW可以有效抵御51%攻击，攻击者必须拥有超过整个系统51%的算力，才有可能篡改系统账本，这使得攻击成本很高。PoW算法同时存在显著的问题。为了发行区块并获取收益，节点需要进行大量的哈希计算。

# 经典共识算法：POS

- 比特币的算力竞争造成了巨大的资源浪费，而且长达10 min的出块时间使其不适合支付、快速交易等商业应用。
- 权益证明（Proof of Stake, PoS）共识算法可以较好地克服PoW的这些弱点，一定程度上解决了PoW算力浪费的问题，并能够缩短达成共识的时间。2011年7月，一位名为“量子力学”的数字货币爱好者在比特币论坛中首次提出了PoS共识算法。PoS共识算法中通过对不同节点特定数量货币的所有权进行统计，选择具有最高权益（Stake）的节点并赋予记账权。如果说PoW的本质是计算能力的比较，PoS的本质就是比较“资源”拥有数量的多少。在对节点所拥有的资源进行测算时，就需要引入币龄(Coin Age)或币天数(Coin Days)的概念。
- 中本聪在2010年提出了币龄（Coinage）的概念，其定义是“虚拟货币”所持有的时间段，即“虚拟货币”金额与用户所持有的时间的数学运算结果（乘积）。类似于将个人钱款存入银行，银行会按照金额和存款时间分配利息。采用PoS的数字资产，区块链会根据用户的币龄来分配相应的权益。而PoS的博弈思想正是持有货币数量越多的人越值得信赖。

# 经典共识算法：POS

- 权益证明解决了工作量证明中浪费资源的问题，同时缩短了共识时间。在PoS中，消耗币龄最多的链为区块链中正确的链，因此若要对区块链进行攻击，则需要控制较多的币龄，大大提升了对区块链进行攻击的成本。

- PoS共识算法更多代表的是一种理念，在不同的应用场景中具有不同的表现形式，其算法也在不断优化。按照其发展顺序，主要包含以下三种PoS共识算法：

- PoW+PoS混合共识

- 由桑尼·金（Sunny King）发行的Peercoin（点点币）是首先采用PoS共识算法的虚拟货币，采用PoW+PoS的方式，同时含有两种共识算法产生的区块。其中，PoW主要用于在初期的挖矿阶段进行货币发行。基于PoW的货币发行方式会随着挖矿难度的上升逐渐减少产币量。随着难度上升，系统中PoW的比重将逐渐降低，系统将过渡到PoS共识。
- 点点币在算法中使用了“币龄”概念，定义为币的数量与币所拥有的天数的乘积，这使得币龄能够反映交易时刻用户所拥有的货币数量。此外，算法还体现出随机化的特点。未使用至少30天的币可以参与下一区块的竞争，越久和越大的币集有更大的可能性能够签名下一区块。而当币的权益被用于签名区块后，币龄将全部清空，且必须等待至少30天才能签署另一区块。同时，为了防止过老或过大的权益控制区块链，寻找下一区块的最大概率在90天后达到最大值，这一过程能够有效保护网络，并保证了随时间推移生成新的币无须消耗大量算力。

# 经典共识算法：POS

## ■ 纯PoS共识

- 2013年发行的NXT（未来币）是使用纯PoS共识的币种，提出了透明锻造的区块生成方式。在NXT中，每个账户对应一个私钥，每个区块有一个生成签名（generationSignature）字段。为了参与区块产生，活跃的账户用其私钥对上一区块的generationSignature字段进行签名，获得一个64字节的签名。然后对这个签名进行SHA256哈希，取结果的前8个字节作为该账户的击中值（hit）。
- 同时，每个账户可以根据当前有效权益计算它自身的目标值，其计算方式为：基准目标值 $\times$ 账户有效货币余额 $\times$ 当前时间与上一区块的时间间隔。hit和目标值进行比较，如果hit小于目标值，那么下一个区块就可以产生。
- 从以上叙述可以看出，通过比较一个账户的hit值和目标值，就可以预测任何一个账户生成区块大致的时间。也就是说，当全网生成一个最新区块后，每个用户锻造下一区块的hit值就已经成为常量，因而用户不需要搜索和挖矿，只需随着时间推移等待hit值小于目标值即可锻造区块。由此可见，在NXT中，上一区块产生后，下一区块由哪个节点锻造就已经确定，全网节点只需等待该节点在正确的时间段生产区块即可。

# 经典共识算法：POS

- PoS共识的扩展算法
  - 基于改进PoS的共识扩展算法目前大多处于研究和实验阶段，需要通过长时间的运行检验来加以测试和完善，主要包含有Tendermint、Casper和Ouroboros等。
- PoS一般对系统硬件没有太高的要求，参与者只需要极小的硬件投入成本就可以参与到全网共识中，这使得能够参与到共识中的受众范围更大。此外，PoS的工作思路类似于金融体系中的“股份”或“存款”，通过质押资产来获取“利息”。这种方式将发行者、持有者等的利益捆绑在一起，能够有效提升整个区块链系统的安全性和稳定性。
- PoS依旧存在一定不足。由于PoS采用权益结余的方式，这很容易产生“富者更富”的马太效应问题；此外，在PoS中，用户可以同时在两个分叉上下注而不会遭受损失，无论哪个分叉在后期被公认为主链，该用户都可以获得奖励而不承受损失。这就会造成“无权益问题”，干扰共识的形成。



# 经典共识算法：DPOS

- 委托权益证明（Delegated Proof of Stake, DPoS又称股份授权证明）通过共识节点的权益投票将区块数据的记账权和区块链参数的配置权赋予少数的代表节点，从而实现民主共识。其解决了共识算法运算速度的问题，使得区块链的交易速度可以与目前的电子支付系统在同一级别上竞争。

- 流程主要包含以下两个部分

- 1. 见证人选举

- 用户节点的投票权重和自身的持币数量成正比，且可以随时投票或撤销。通常，系统会选择得票数最高且愿意成为见证人的前N个节点进入“董事会”（N一般为奇数），并由这些节点负责生产区块。显然，DPoS是一种弱中心化的共识算法：当见证人的数量越小时，系统的中心化程度就越高。但这种弱中心化的方式能够有效帮助系统提升运行效率，同时减少能源浪费。
    - 见证人在DPoS共识过程中的主要职责与PoW中的矿工相似，主要负责侦听网络中广播的交易、对交易有效性进行验证并打包区块、广播区块并更新主链等。与矿工“挖矿”并获取收益类似，见证人也会获得一定的系统奖励作为工作回报。同时，作为区块链代表节点，见证人必须上缴保证金作为个人工作责任的担保。如果见证人未能有效履行相关职责，用户节点可以撤回投票并将该见证人从“董事会”撤出。

# 经典共识算法：DPOS

## ■ 流程主要包含以下两个部分

### ■ 2. 区块生成

- DPoS共识过程中，见证人节点按照预先定义的顺序轮流生产区块。对于每一个见证人节点，都会分配一个固定的时间段用于生成区块。如果见证人没有在其时间段内生产区块，那么时段之后该见证人将会被跳过，由下一个见证人生产下一个区块，以此循环。在循环过程中，每产生N个区块（N为见证人数量），见证人的排序会被打乱。这种“洗牌”操作会通过随机的方式确保每当形成多个拥有相同数量见证人的分叉时，平局都会被打破。
- DPoS共识算法有着较快的处理速度，做到了秒级的共识验证，比PoW共识算法有了较大的提高。基于DPoS共识算法的平台性能较高，且不需要竞争挖矿等。但同时，该共识算法的缺点也很明显。用户通过选举代表节点来负责区块生成，就会造成弱中心化的局面。严格来说，只有可以出块的超级节点可以参与共识，而后续参与者则很难成为超级节点并参与到生成区块的过程中。因此，有不少人质疑这种共识算法的开放程度。



# 经典共识算法：PBFT

- 实用拜占庭容错算法（Practical Byzantine Fault Tolerance, PBFT）是由米格尔·卡斯特罗（Miguel Castro）和芭芭拉·里斯科夫（Barbara Liskov）在1999年于第三届操作系统设计实现研讨会上提出的，相比于需要指数级运算的早期拜占庭容错算法，PBFT将算法复杂度降低到多项式水平，解决了效率问题，使工程使用成为可能。
- 与RAFT共识算法不同，PBFT假设共识过程的运行环境是异步分布式网络，且可能存在恶意发送错误信息的拜占庭节点。PBFT通过要求整个系统共同维护一个状态，即实现节点协约一致性来确保系统安全性。
- PBFT共识算法协议
  - PBFT共识算法的每一个视图下都有一个主节点，其他为从节点。主节点负责对请求进行排序，按顺序发送给其他的从节点。如果从节点检查到主节点出现异常，就会触发视图切换机制更换下一个编号的节点为主节点，进入新的视图。
  - PBFT共识算法包括三个协议：一致性协议、检查点协议、视图更换协议。
  - 1. 一致性协议
    - PBFT的主要协议，通过两轮投票，使系统的各个节点达成共识并出块。主节点负责将客户端的请求排序，从节点按照主节点提供的顺序执行请求。整个过程分为请求(Request)、预准备(Pre-prepare)、准备(Prepare)、确认(Commit)、回复(Reply)五个阶段。

# 经典共识算法：PBFT

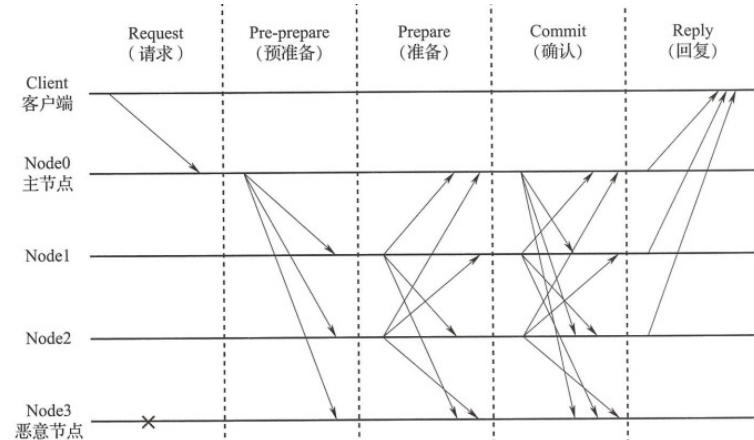


图 3-10 PBFT 共识过程

## ■ 1. 一致性协议

- 假定整个系统中的拜占庭节点个数为 $f$ 个，而服务节点总数为 $3f+1$ 个。整个共识过程可以分为5个阶段，每个阶段的具体内容如下：
  - 请求阶段 (Request)：客户端向主节点发送请求，其中包含需要执行的操作，时间戳信息，客户端ID及签名，用于确认客户端权限。
  - 预准备阶段 (Pre-Prepare)：当主节点接收到请求后会对消息进行签名并为请求分配一个唯一的序号 $n$ ，并将该序号和客户端请求信息广播给其他节点。主节点签名能够便于其他节点认证主节点身份。
  - 准备阶段 (Prepare)：所有的备份节点都收到由主节点发来的Pre-Prepare消息后，将一个包含当前视图号 $v$ ，消息序号 $n$ ，消息摘要的Prepare信息发给其他所有节点。如果一个节点收到了 $2f$ 个以上相同的Prepare消息，则进入下一阶段并将该消息标记为Prepared状态。
  - 提交阶段 (Commit)：每个节点广播包含视图号 $v$ ，消息序号 $n$ 的Commit消息。当节点收到 $2f$ 个相同的commit消息，且序号小于 $n$ 的消息都已经执行，则当前消息会被执行。
  - 回复阶段 (Response)：所有节点都将执行结果返回给客户端。
- PBFT能在异步环境下，容忍不足 $1/3$ 的总数节点是拜占庭节点，并在这一条件下达成共识。此外，在PBFT中，消息的发送者是经过认证的（签名），发送消息的身份可以被确认，节点是否发送消息也可以进行验证。

# 经典共识算法：PBFT

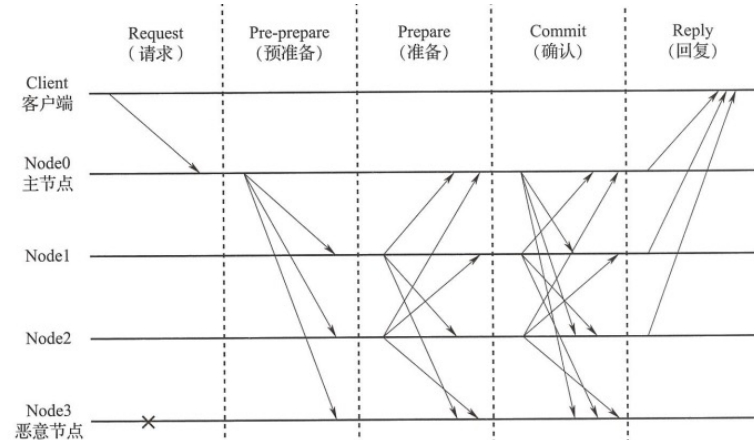


图 3-10 PBFT 共识过程

## ■ 2. 检查点协议

- 检查点协议是一个辅助协议，用于解决分布式环境下服务器时间同步问题，清理日志，解决资源占用。

## ■ 3. 视图更换协议

- 主节点在整个共识的流程中，充当着中心的角色，当主节点出现故障，或者主节点是恶意节点时，为了保证整个系统的可用性，需要使用视图更换协议更换主节点。视图的更换需要多方达成共识，然后可以进行切换。
- PBFT的应用场景较为广泛，在区块链领域中，PBFT一般适用于对一致性有较高要求的联盟链和私有链场景，即需要保证系统各节点的强一致性。如，在IBM主导的区块链超级账本项目（Hyperledger Fabric）中，PBFT是一个可选的共识算法，Fabric0.6版本就自带了PBFT共识算法。
- PBFT算法是一种静态的共识算法，适用于节点数量有限的情况，对于节点数量不断变化的动态网络，算法的工作效率不是很高。

# 共识算法的发展：混合共识算法

- BFT类共识算法实现了高吞吐量和及时一致性，但因为随节点数平方上升的消息复杂度、固定的节点列表、已知的节点身份、主节点的性能瓶颈等原因，更多应用于联盟链或私有链场景中。相比之下 PoX(如 PoW, PoS)类共识算法提供了良好的网络可扩展性，可用于公有链环境。PoX类算法达成网络节点的一致性是以低吞吐量和高时延为代价的，同样也有缺点。
- 为了在公有链中兼顾高吞吐性能和网络可扩展性，一种做法是将BFT与公有链类算法结合，将 BFT 类共识引入到公有链的业务场景中。
  - 由丹尼尔.拉里默开发的石墨烯“全家桶”平台，包括比特股、斯蒂姆、企业操作系统(Enterprise Operating System, EOS)等，使用 PoS+BFT 的组合作为其共识算法。持有者以投票等方式选出自己支持的代表，并由这些代表组成的见证人网络通过 BFT的方式进行共识。
  - 授权拜占庭容错(Delegated Byzantine Fault Tolerance, dBFT)是 NEO 采用的共识算法，是一种通过代理投票实现大规模参与的共识算法。节点可通过投票选出所支持的代理，然后代理再通过类PBFT共识算法达成共识。dBFT具有快速、扩展性好等特点。

# 共识算法的发展：PoW的改进

- 公有链交易需求不断增加，但其交易吞吐量却被严格的限制在区块大小除以区块间隔的范围内，公有链的可扩展性问题越来越突出。系统最多只能实现适度的吞吐量，从每秒约3个增加到每秒约7个交易，与维萨(VISA)每秒30 000 笔交易相差甚远。

- 下一代比特币需要消除上述可扩展性限制，通过使比特币达到网络条件允许的最高吞吐量来解决可伸缩性瓶颈。

- 2015年，康奈尔大学的学者提出了Bitcoin-NG共识算法，以比特币信任模型为基础，将比特币的共识过程改进为领导者选举和交易序列化两部分，在不牺牲其他性能的基础下有效改善了原生算法的延迟问题。在Bitcoin-NG共识算法中，时间被以时间段的形式进行划分，算法按照时间段为单位进行领导者选举。在每个时间段内选出的一位领导者将负责此时间段内创建区块的任务。Bitcoin-NG共带有两种区块：关键区块与微区块，其中，关键区块用于领导者选举过程，其生成过程类似于前述的PoW共识算法区块头的哈希值必须小于特定目标值，该区块才会被标记为有效。一旦某个节点成功生成了关键区块，其就成为该时间段的领导者，并开始以固定速率生成多个微区块。微区块包含账本记录，即对交易信息进行记录。与PoW共识算法不同，微区块的生成不需要工作量证明，因此领导者可以快速生成多个微区块。

# 共识算法的发展：PoS的改进

■在原生PoS共识算法中，存在“Nothing to Stake”的问题，也就是节点进行恶意操作的代价很低的问题。针对这一不足，Casper投注共识应运而生。Casper是一种基于保证金的经济激励共识算法，协议中的节点必须首先缴纳保证金，才可以参与出块和共识过程。Casper算法就是通过对保证金的控制来约束节点的行为，如果节点做出任何无效或恶意操作，其对应保证金将会被罚没，参与共识的权利也会被取消，参与共识节点的列表会随着保证金不断地锁定、罚没而产生变动。

■从算法的角度来看，PoS 改进共识算法主要有两种类型

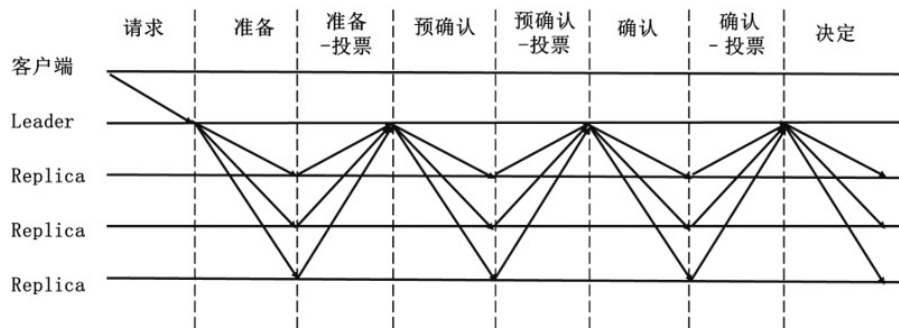
- 基于链的权益证明：在每个时隙(例如，每10s的时间段可能是一个时隙)中伪随机地选择一个验证者，并向该验证者分配创建单个块的权限，该块必须指向先前的块(通常是在先前最长的链末尾的块)，因此随着时间的推移，大多数块会汇聚为一条不断增长的链。
- BFT风格的权益证明：验证者会被随机分配提议区块的权利，但要通过多个回合就哪一个区块是规范的达成共识。在每个回合中，所有验证者都会针对某个特定区块发起“投票”，在该回合结束时，所有诚实和在线的验证者都需要永久地投票确定任何给定的块是否属于该链的一部分。需要注意的是，对区块的共识不取决于最长链。



# 共识算法的发展：PBFT的改进

■ PBFT共识算法因为有诸多优点，如共识效率高、不产生分叉、容错性强(容错小于  $1/3$  的恶意节点)、无须消耗算力资源等，被当作联盟链的首选共识算法。Linux 基金会超级账本项目下的Fabric和国内诸多的联盟链项目都采用了PBFT作为其共识算法。

■ 2019年，Maofan Yin等人提出了线性响应BFT类共识算法——HotStuff。HotStuff与PBFT共识算法类似，是一种基于视图的共识协议。在每个共识单元中，都由唯一确定的Leader节点来领导共识过程，并采用三段确认流程来达成全网一致，然后切换到下一个共识单元中，由新的Leader节点继续进行新一轮的共识。相比较于传统的BFT类共识算法的特点，HotStuff共识算法实现了线性的共识模式，提高了共识的效率。



HotStuff 共识流程图



# 共识算法的发展：跨链共识

- 随着区块链技术被广泛的接受和应用，出现了大量不同用途和性质的区块链平台和网络，这些高度异构的链之间是相互独立和封闭的，各自的数据和价值也都被限制在链本身，难以发生更大范围内的共享和流通。要打破这种局面，必须深入到跨链技术的研究中去，思考如何在跨链的情况下达成链间的共识。
- **跨链**技术是为了解决不同区块链之间连接、交互的问题，支持区块链账本之间的资产和数据的转移和连通，在不同性质的链间实现价值流通。目前跨链技术共识算法取得了一定的进展，已经发展出了**侧链机制**、**公证人机制**、**中继机制**等方法。需要解决跨链技术在共识过程中的难点包括跨链交易的可验证性和安全性。

# 总结

| 类别     | 共识算法                          | 特点                     | 适用场景            |
|--------|-------------------------------|------------------------|-----------------|
| 工作量证明  | PoW                           | 通过计算竞争获取记账权；安全但能耗高、速度慢 | 比特币等公有链         |
| 权益证明   | PoS                           | 通过持币量和时间决定权利；更节能       | 以太坊2.0、波卡等      |
| 委托权益证明 | DPoS                          | 通过投票选出代表记账；效率高但中心化风险   | EOS、Steem       |
| 拜占庭容错类 | PBFT                          | 适用于已知节点系统，效率高、容错强      | 联盟链/私有链         |
| 拜占庭类改进 | HotStuff                      | 基于视图线性推进共识，简化流程提高性能    | Libra等系统        |
| 混合算法   | PoS+PBFT/dBFT                 | 结合不同机制的优点              | NEO、EOS         |
| 扩展算法   | Casper, Tendermint, Ouroboros | 引入质押与惩罚机制，增强安全性        | 公链PoS演进方向       |
| 跨链共识   | 中继、公证人、侧链等机制                  | 用于多链协作，达成跨链数据一致性       | Cosmos、Polkadot |